

Feedback from QUEST

Problems regarding the MDP model:

- The authors introduce an MDP, but do not use it to find an optimal policy. Instead, they just compare 4 policies. It seems the MDP formulation is only used to define the max-capacity and min-cascade strategy.
 - We are not finding the optimal policy
 - We demonstrated 4 policies to show the model is working correctly.
- Is the model an MDP if your repair times are not geometric (as in the case-study)?
Also, does the probability $d_{\{i,j\}}$ apply once, that is, when i fails, or each time instance as long as component i is failed. I assume the latter.
 - So far I thought about two solution:
 - Expand the state of the components to a sequence of repair steps to operational
 - Or make the model a semi-Markov decision process
 - I think my model is much more similar to Semi Markov decision process (SMDP) at this point
 - The repair actions would be the part requires memory of the starting point and make specified jumps at completion.
 - Look at Markov regenerative process, Phase-Type Expansion

- The probability of fault propagation should use $(1/n * D)^k$ instead from the following section of Koosha's work:

2.3. Quantification of Interdependencies

Assuming that the direct influence matrix $\mathbf{D}$ is known, we can explore dependency of components in multiple **time** steps. Specifically, we are interested in the k^{th} -level influence matrix, which represents the influence that components have on each other over exactly k **time** steps – in contrast to $\mathbf{D}$, where the influence exerted over a single **time** step is captured. To this end, we first normalize $\mathbf{D}$ by dividing it by n to ensure that $\sum_{j=1}^n d_{ij} \leq 1$. Matrix $(\frac{1}{n}\mathbf{D})^k$ is hence guaranteed to have an upper bound, and represents the k^{th} -level influence matrix. The *total influence matrix*, $\mathbf{T}$, can be computed as shown in Equation (4).

$$\begin{aligned}\mathbf{T} &= [t_{ij}] = \mathbf{V} \circ \sum_{k=1}^{\infty} \left(\frac{1}{n}\mathbf{D}\right)^k \\ \mathbf{V} &= [v_{ij}] \\ v_{ij} &= \begin{cases} \frac{n+1}{n}, & i \neq j; \\ \frac{n+1}{n-1}, & i = j. \end{cases}, \quad 1 \leq i, j \leq n\end{aligned}\quad (4)$$

In Equation (4), $\circ$ represents the entrywise product and t_{ij} shows the degree by which component j can be influenced by a failure in component i in any number of **time** steps, which reveals indirect influences. Note that the matrix $\mathbf{V}$ is used to scale t_{ij} to $[0, 1]$ range.

- Another benefit of switching to SMDP would be memorizing the starting point of each failure to track the time-based consequences.
- You state that you "solve" the MDP. What do you mean by this? It has a huge number of states, 2^{27} for the component vector only.
 - I believe the review is referring to the sentence "The MDP was formulated and solved in Python 3 on Google colab"
 - I should change "solve" to "implemented"
 - Yes, it is a massive model to work with pencil and paper, so we must use python as it supports massive memory space from Google cloud.
- Are remaining repair times part of the state?
 - Yes, it identifies the remaining time for crews to be "busy" under a repair task. Mainly for time tracking, not affecting the calculation of SCI or anything else.

- What solution method is used to find a policy? Value/policy iteration? Monte Carlo simulation? (The authors only mention Python3 and Google Colab)
 - The recovery schemes should be policies. Policy is a function that specifies the action that the decision maker will choose when in a particular state. Our testing policies are adapted from recent studies and theories. The optimal policy is found using Monte Carlo simulation (Mentioned in section 4.2 Failure cases) by comparing the results of the recovery process.
 - Implementation diagram: [simulation process.png](#)

Input	Software	Output
IEEE-14 smart grid data, historical failure events	MATLAB & PSAT	Dependency index, SCI look-up table
Dependency index, SCI look-up table, repair time of components, number of crews, scheme to use,	Python	System performance during Recovery process (SCI over time), first recovery time, number of cascades

- How well does simulation perform when its allowed time is the same as the solution method proposed by the authors?
 - I am not quite sure what does the reviewer meant by “allowed time” and “the solution method”, how can time be the same as a method?
- Regarding recovery times longer than one time step, how is the Markov property maintained?
 - I believe the reviewer is referring to “Memoryless property”
 - We use repair time to track the repair progress of a component, decrementing by 1 in each time step. It does seem like a “memory” property as the model remembers the repair progress, but it’s basically a one way transition from a starting integer to 0, once per time step.
- Do you save the remaining steps until recovery in the state space of the MDP? If so, doesn’t this then lead to quite an explosion of the state space, which might be the reason that standard planning/RL techniques are not applicable after all?
 - The current version of the code does not save the steps until recovery; it will lead to quite an explosion for sure. I only save the SCI value for each step until recovery, because SCI is designed to reflect the system status on a scale between 0 and 1.
 - For AI techniques like reinforcement learning, ANN, they do need to remember the states and their choices. For example, the simplest machine learning technique, is Q-learning. Which creates a table matching all the

possible states and branching out all the possible actions. It will be rating the actions with rewards to see which choice was the best from the training data. That would create a Q-table with at least 2^{27} floating point value (assume only one branch). That is huge, and IEEE-14 smart grid is not a large system.

- Although it's difficult, but I believe the creation of reinforcement techniques, AI, for recovery task scheduling is possible. Our work focuses on creation of the model, a stage for testing the schemes, and identifying the best scheme. Creation of new schemes is part of the future direction of our research.

Problems regarding recovery schemes:

- No info is given on the details of the pre-computed scheme.
 - I remember having an explanation of how pre-computed scheme was made for our case study, it was basically a **static** Min-Cascade scheme. That finds the highest to lowest cascading index of each component in the dependency matrix. But it's not there.
- I would guess that the best policy of the 4 considered might depend on the numbers used? For instance, if there is almost no propagation of failures, then max-capacity makes sense. If there is lots of propagation you might go for min-cascade.
 - From our observations, the Min-cascade out-performed others. The max-capacity was adapted from the nearest work, and it was proven to be the best policy in post-hazard (no propagation) situations in their paper.
 - "I would guess that the best policy of the 4 considered might depend on the numbers used" For any chaotic mathematical model, a little change in the initial conditions can alter the results. For us, the uncertainty in cascading failure and repair time make up the chaotic behavior in our model. What we are doing in the case study is finding the convergence of system behavior with 4 schemes using Monte Carlo simulations and identifying the one scheme that most likely to produce the best results.
 - Best performance scheme is based on actual number used, and the main contribution is to allow comparison
- Have you thought about combining your two solution schemes (Max-capacity and Min-cascade) in some way? As both things are shown to be important for optimizing the performance it seems like a natural idea to look for schemes that combine both metrics (e.g., via an hyper parameter, via some clever weighting, via some other multi-objective-technique,...)

- I have thought about Multidimensional figure of merits for recovery decision support in the future direction. Service capacity could be one figure of merit, and cascade mitigation could be another.
- It seems quite surprising to me that the random scheme performs that good in the experiment with five crews (Figure 6, Table 5 & 6). While it seems reasonable that it gets better with increasing crew size (also better relative to the performance increase of the other schemes), it is remarkable that it is almost as good as your proposed Max-capacity and Min-cascade schemes. Do you have any explanations thereof?
 - When there are more available crews than failed components, there is nothing to schedule, no strategy at all. You can always find an idling crew to assign a repair task.

Problems regarding the case study

- The case study simply assumes uniform restore times, not real data. To what extent are the results/insights affected?
 - The assumption of uniformly distribution of restore times are used to mimic the uncertainty of time when repairing a component. In the Monte Carlo simulation, this technique will impact the result with randomness. It will scatter the system restoration time in similar cases and finds the upper or lower bound of the possible outcome.
 - The assumption of uniformly distribution can also be replaced by other distributions, such as exponential distributions, Weibull distribution, but we have to have evidence that the recovery time of the component is following that distribution. Uniformly distribution is used here only for illustrative purposes.
 - The repair time does not has to be uniformly distributed

Problems regarding the paper

- The main conclusion is "The results showed that each dynamic scheme achieved the objective for which it was designed. The Min-cascade scheme was the best overall, in terms of both time to first recovery and number of cascading failures." The former is not very insightful and the latter may depend on the numbers used.
 - We might have to switch the conclusion to something related to the model-based decision support.
 - "The results showed that each dynamic scheme achieved the objective for which it was designed. Which showing, the model we present was able to predict the recovery process of a give system during cascading failure event.

The behavior of the repair crews was following the designed objectives of each scheme. We are able to compare the results and identify the best scheme for the given network.” something like this?

- The paper is well presented, but given the strong model assumptions, I do not see much practical value in the paper.
 - Should we relax some assumptions? I am not sure which assumption is too strong. Here is the list of assumptions:
 - Fully observable, estimation of repair time are both available with Ameren from my interview
 - System is fully recoverable, has to be there for a recovery model, or else the model is working on impossible tasks. I guess we can say that we exclude unrecoverable component from our model.
 - Other values of the paper we might want to add:
 - the scalability of the model. Although the state space of the model is 2^N , but it does not require storing the entire state space for simulation. It only requires the information about current state to drive the next state (Markov property). The largest memory requirement for running the simulator is actually the dependency matrix ($N \times N$) floating values.
 - the compatibility of the model. our model can apply but not limit to smart grid, it can also be applied to other complex networked system, such as water, petrol gas, mobile network systems, any system with multiple layer interdependencies.
- Other works use MDPs or reinforcement learning to optimize a repair strategy, while other failures may occur.
 - Adapting reinforcement learning to our model could be difficult. I would spend time on it but I am not sure how long will it take me. I have studied reinforcement learning and Artificial neural network. The main topic of our study was Q-learning and image recognition. They are not complex enough to schedule recovery tasks with long-term consequences (fault propagation).
 - The uniqueness between my work and the other MDPs with fault occurrence
- The conclusions do not appear to be particularly significant for the application domain. As the authors note, some model parameters are assumed for "illustrative purposes" (e.g., uniform recovery times); similarly, the superior performance of MaxCapacity and MinCascade is not surprising, given their dynamic nature.
 - I could adapt a few more dynamic strategies to compare.
 - Instead of uniform recovery time, other types of distribution (exponential, Weibull) can also be used for our case study.

- the development crucially bases on previous works of the same authors, that are almost only referenced, but not really summarized/recapped in the present paper itself (which I think is basically fine, however I also think that there would have been space to recap the important ideas at least briefly).
 - Better summary of previous work
- the setup of the case study is described in a very large detail, that should be possible to dense, the Figures 3 and 4 are very similar, or also the remark on how crucial it is to consider the cascading errors is repeated very, one might say intrusive, often
 - We could shorten some case study setups for more space
- While I see some interesting contributions, mainly in terms of modeling the recovery task with cascading failures and the developed solution schemes, I am not sure whether they reach far enough and are novel enough for a full paper yet. For instance, I would definitively think that the experimental evaluation should be broader and not only one instance of a smart grid should have been tested. Therefore, it seems unclear to me by now, whether the presented results have a chance to hold in at least some generality or only for this one specific example.
 - we can include the results from IEEE-9, or we can work on a new grid, something like IEEE-57
 - A bottleneck for doing larger grids will be the SCI look-up table. For IEEE-14 smart grid, I have to reserve a memory space of 2^{27} floating values. If we switch to 57 or 108, it would be impossible to use the lookup table. We have to use some other ways to analyze the system performance/capacity.
- I was also wondering how standard methods for solving MDPs from planning or reinforcement learning perform in comparison to the presented solution schemes. On the first glance I think that learning methods should be applicable in the present use case as the MDP seems to be sampled in the papers solution pipeline anyways (e.g. in the power flow analysis). If not applicable for some good reason, I think that this should be argued in the paper.
 - The learning methods will require large trials and training dataset, which could be time-consuming. But I am willing to give it a try.
 - Next step in the research

Outstanding papers to review

- Q-Learning-Based Resilience Assessment of Weakly Coupled Cyber-Physical Power Systems <https://ieeexplore.ieee.org/document/10738835>
 - A graph model is created to analyze the recovery process of a CPS, edges are power line, and nodes are generators, buses, and loads. Using probability to

determine if a node or edge will fail due to cascade, similar to our work. Failed nodes will be removed from the graph model. Repair begins after all the cascade has been done.

- Did not talk about how time flows with the model. Where do they get the repair time for each component, how time was assigned to repair task, how many crews.
- Repair Crew Routing for Infrastructure Network Restoration under Incomplete Information <https://arxiv.org/pdf/2505.05297>
 - A MDP model is created to analyze the recovery of one failure in a CPS along with its cascading failures. The location of the fault is hidden, the crew has to examine the component to locate the fault.
 - Assumed to know the exact repair time. Although repair time is not their main concern. They care about navigation of one crew, locating failure and repair them.
- Automatic Recovery Using Bounded Partially Observable Markov Decision Processes <https://ieeexplore.ieee.org/document/1633533>
 - POMDP created to schedule the recover tasks, and predict the lower bound of the system performance.
 - No cascading failure.